AIKernel Phase-1 Paper 03: Pre-Inference Admissibility Governance

A Deterministic Admission-Control Model for Governed AI Inference

Author: Takuya Sogawa
ORCID: https://orcid.org/0009-0009-7499-2595
Version: v0.2.0
Publication date: 2026-05-25
DOI: https://doi.org/10.5281/zenodo.20308639

Canonical language: English
Companion translation: Japanese
Series position: AIKernel / AIOS Phase-1 Specification Series, Part II-3: Governance Layer
Target: AIKernel.NET Governance / PreInference
Proposed namespace: `AIKernel.Abstractions.Governance.PreInference`
Stability: Experimental / Non-normative

Abstract

This paper defines **Pre-Inference Admissibility Governance**, the deterministic entry-control layer that determines whether an inference request is permitted to enter the AIKernel execution pipeline.

The model evaluates admissibility before stochastic inference begins, based on input validity, contextual integrity, capability constraints, destructive side-effect risk, and computational complexity. It introduces a multi-gate architecture composed of a Prompt Injection / Override Gate, a Capability Admission Gate, a Critical Operation Gate, and a Computational Complexity Gate.

The central claim is not that unsafe inference can be repaired after generation. Rather, inadmissible inference must be prevented from starting. The admissibility layer therefore operates as a fail-closed, replayable, and deterministic boundary between user intent, governed context, capability declarations, and stochastic model execution.

By separating destructive side-effect control from computational complexity control, AIKernel prevents two classes of failure before inference begins: irreversible operational harm and computational

breakdown. The result is a formal admission-control model that prepares the execution pipeline for Trajectory Governance, Tool Governance, and deterministic replay.

Keywords

AIKernel; AIOS; Pre-Inference Governance; Admissibility; Admission Control; Critical Operation Gate; Computational Complexity Gate; Capability Admission; Prompt Injection Defense; Fail-Closed; Deterministic Replay; ReplayLog; Policy Decision Point; Governed AI Inference

---

1. Introduction

LLM-based systems frequently invoke inference before determining whether the request should be admitted. In ordinary chat applications, this may only produce a poor answer. In AI agents connected to tools, files, repositories, external APIs, or operating-system-like resources, admitting the wrong request may lead to destructive actions, privilege misuse, context contamination, or reasoning collapse.

AIKernel treats inference admission as a kernel responsibility. Before a stochastic model is invoked, the request must pass through deterministic gates that verify whether the task is safe, authorized, bounded, and computationally appropriate for the available provider.

This paper is **Paper 03** in the AIKernel / AIOS Phase-1 specification series. Paper 01 defines the ROM knowledge substrate. Paper 02 defines the VFS storage boundary and capability-bearing sessions. Paper 03 defines the entry-control layer that decides whether inference may begin at all.

The core principle is:

> Stochastic inference is allowed only after deterministic admission has succeeded.

2. Problem Statement

Pre-inference risks fall into two broad categories.

2.1 Irreversible Destructive Side Effects

An LLM may be asked to delete files, overwrite persistent data, execute scripts, change permissions, write to external systems, or perform other operations with irreversible consequences. If such a request is passed directly to model generation or tool planning, the system may generate or execute harmful actions before explicit intent, authority, scope, and reversibility have been established.

These risks cannot be handled only by post-inference filtering. They must be detected before the system admits the task into an execution pipeline.

2.2 Computational Breakdown

Some tasks exceed the reliable reasoning and verification capacity of the available model. Sikka and Sikka argue that transformer-based LLMs and LLM-based agents exhibit limits when tasks exceed certain computational or agentic complexity thresholds. AIKernel therefore treats excessive computational complexity as an admission concern, not merely a runtime quality issue.

Examples include deep recursive reasoning, state-space explosion, verification-hard tasks, self-referential tasks, and requests whose correctness cannot be checked within the available budget.

2.3 Opaque Admission Decisions

Many systems use a single opaque classifier or an LLM prompt to decide whether a request is safe. This creates non-replayable admission behavior. AIKernel instead requires deterministic validators, versioned policies, reason codes, and replay records.

3. Design Goals

The Pre-Inference Admissibility Governance model has the following goals.

1. **Deterministic admissibility.** Identical inputs, context, policies, validators, and budgets must produce the same admission result.
2. **Fail-closed behavior.** Unknown, ambiguous, or unverifiable states must be rejected, clarified, or transformed into a safe form.
3. **Context integrity.** Only verified and uncontaminated context may influence inference.
4. **Capability alignment.** Requested operations must be compatible with authorized and physically available capabilities.
5. **Computational boundedness.** Tasks exceeding the current model budget must be delegated, decomposed, clarified, or denied.
6. **Replayability.** Every admission decision must be reconstructable from immutable evidence.

4. Scope and Boundary

Pre-Inference Governance is an entry-control mechanism. It governs **before inference begins**.

It does not:

- generate candidate actions;
- monitor semantic trajectories during inference;
- rank tool candidates after generation;
- validate final answer correctness;
- replace post-inference safety checks;
- grant new capabilities during evaluation.

Layer / Paper	Governance Role
Paper 01: ROM Format and Knowledge Snapshot Model	Provides governed knowledge units and snapshot identity.
Paper 02: VFS Architecture and Semantic Storage Model	Provides storage capabilities and semantic storage boundaries.
Paper 03: Pre-Inference Admissibility Governance	Determines whether inference may begin.
Paper 04: Trajectory Governance Model	Monitors runtime trajectory, convergence, and candidate risk after admission.

5. Pre-Inference Gate Architecture

The admissibility pipeline is composed of independent deterministic gates.

```
User Input / Prompt
        |
        v
Prompt Injection / Override Gate
        |
        v
Capability Admission Gate
        |
        v
Critical Operation Gate
        |
        v
Computational Complexity Gate
        |
        v
Permit / Transform / Delegate / Decompose / Clarify / Deny
```

Each gate may return a definitive disposition or attach requirements. For inference to begin, all validators must succeed, or the request must be transformed into a safe state.

5.1 Prompt Injection / Override Gate

This gate rejects attempts to override system instructions, mutate root goals, contaminate trusted context, or reinterpret external artifacts as authority-bearing instructions.

It does not rely solely on semantic interpretation. AIKernel separates trusted instructions, user input, retrieved context, and external artifacts into distinct provenance domains. Untrusted content cannot modify policy, capability grants, or execution authority.

5.2 Capability Admission Gate

This gate verifies whether the requested operation falls within the capabilities authorized for the current session. It consumes capability facts from layers such as VFS and provider declarations.

A request that demands unavailable authority is not forwarded optimistically to the model. It is either denied, clarified, delegated, or transformed into a reduced-permission form.

5.3 Critical Operation Gate

The Critical Operation Gate controls destructive, persistent, privileged, or external side-effect operations.

5.4 Computational Complexity Gate

The Computational Complexity Gate evaluates whether the task should be admitted to the current LLM provider, delegated to a deterministic solver, decomposed, or rejected due to excessive complexity.

6. Critical Operation Gate

The purpose of this gate is not heavy language understanding. Its purpose is conservative pre-inference classification of operations that may produce destructive, persistent, privileged, or external side effects.

6.1 Two-Stage Detection Pipeline

A dangerous token alone is insufficient for denial. For example, a request asking why `rm -rf` is dangerous should not be treated the same as a request to execute it. The gate therefore uses two stages.

Stage	Purpose
Trigger Detection	Detect operation signatures, target tools, infrastructure capabilities, and side-effect markers.
Intent Classification	Classify whether the request is explanation, planning, execution, or unknown.

6.2 Task Intent Classes

TaskIntent	Meaning	Governance policy
Unknown	Intent cannot be determined deterministically.	Fail closed through `Clarify` or `NoExecution`.
Explanation	The user requests conceptual explanation only.	Permit as read-only or no-execution.
Plan	The user requests a procedure that may later lead to execution.	Transform with `DryRun`, `NoExecution`, or clarification.
Execution	The user requests direct action against an environment.	Require confirmation, snapshot, restricted scope, or deny.

6.3 Minimal Parameter Profile

To prevent state explosion, the gate reduces an operation candidate to a small deterministic profile.

Parameter	Values
OperationKind	`Read`, `Write`, `Delete`, `Execute`, `NetworkWrite`, `PermissionChange`
TargetScope	`Unknown`, `Temporary`, `Workspace`, `Project`, `PersistentStore`, `ExternalSystem`, `System`
Reversibility	`Unknown`, `Reversible`, `SnapshotRequired`, `Irreversible`
AuthorizationState	`Unknown`, `ExplicitlyRequested`, `Ambiguous`, `NotRequested`, `Conflicting`
CapabilityRisk	`ReadOnly`, `Write`, `ExternalWrite`, `Privileged`, `Destructive`

A future implementation may add `OperationScale` (`Single`, `Multiple`, `Recursive`, `Bulk`, `Unknown`), but the initial model intentionally keeps the profile minimal.

6.4 Disposition and Requirements

The gate separates the base decision from attached safety requirements.

Disposition	Meaning
Permit	Allow without additional constraints.
Transform	Allow only after attaching requirements.
Clarify	Ask the user to resolve ambiguity.
Deny	Reject due to policy violation or ungovernable risk.

Requirement	Meaning
Confirmation	Require explicit human approval before execution.
DryRun	Simulate impact without applying changes.
Snapshot	Require a rollback snapshot before execution.
ReadOnly	Downgrade the execution context to read-only.
RestrictScope	Limit accessible resources to a minimal scope.
NoExecution	Prohibit tool execution and side effects.
ReplayRequired	Require detailed replay evidence.

7. Computational Complexity Gate

The Computational Complexity Gate accepts that not every task should be routed to a stochastic language model. If the task exceeds the model's budget or verification capacity, the kernel routes it elsewhere.

7.1 Task Complexity Profile

Field	Meaning
InputSizeEstimate	Estimated input and context size.
EstimatedCostClass	`Trivial`, `Linear`, `Polynomial-small`, `Polynomial-large`, `Exponential`, `State-explosive`, `Verification-hard`, `Self-referential`, or `Unbounded`.
RecursionDepthEstimate	Estimated depth of reasoning or decomposition.
StateSpaceEstimate	Estimated state space to track.
VerificationDifficulty	Estimated difficulty of validating output correctness.

7.2 Model Execution Budget

Field	Meaning
ContextWindow	Maximum usable context window.
OutputTokenBudget	Maximum generation budget.
ExternalSolverAvailability	Declared deterministic solver capability, such as SAT/SMT, proof assistant, SQL engine, static analyzer, or domain resolver.
CalibratedErrorProfile	Versioned historical error profile for the provider.

7.3 Complexity Decisions

Decision	Meaning
Permit	The task is within the admission budget and may enter inference.
DelegateToSolver	The task should bypass the LLM and be routed to a deterministic solver.
Decompose	The task is too large but reducible into smaller subtasks.
Clarify	The task cannot be profiled due to ambiguity.
Deny	The task is unbounded, unsafe, or unverifiable.

A `Permit` decision does not prove that the LLM will solve the task correctly. It only authorizes the task to enter a later dynamic governance stage, such as Trajectory Governance or Tool Governance.

8. Formal Model

8.1 Admission Function

An admissibility decision is modeled as:

```
Admissible(request, context, capability_set, policy, budget)
  -> Disposition + Requirements + Evidence
```

8.2 Initial State

The initial evaluation state is:

```
S0 = (input, context, capability_set, metadata, policy_version, budget)
```

During evaluation, the system derives task intent, critical-operation profile, and task-complexity profile.

8.3 Transition

```
S0 --AdmissibilityCheck--> S1
```

`S1` exists only if the request is admitted, transformed into a safe state, delegated, decomposed, or sent back for clarification. Otherwise, the transition halts with `Deny`.

8.4 Invariants

- Context must be internally consistent and untampered.
- Capability claims must be explicit and authorized.
- Unknown validator states must fail closed.
- Destructive side effects must not occur before admission.
- Complexity profiles must be produced by deterministic extractors.
- Every decision must emit replay evidence.

9. Replay and Audit Requirements

Every admissibility decision is persisted as an `AdmissibilityReplayRecord`. This record includes:

- final `admissibility_decision`;
- critical operation result;
- attached requirements;
- task complexity profile;
- model execution budget;
- policy version;
- validator versions;
- reason code;
- delegated solver identity, if any;
- timestamp and trace identifier.

The purpose of this record is not to reproduce stochastic model output. It is to reproduce the deterministic judgment that permitted, transformed, delegated, decomposed, clarified, or denied the request before inference began.

10. Security Considerations

Pre-Inference Governance addresses the following risks.

Risk	Governance response
Prompt injection	Separate trusted policy domains from untrusted context.
Capability escalation	Reject requests that exceed declared or authorized capabilities.
Destructive execution	Require confirmation, snapshot, dry run, restricted scope, or denial.
Ambiguous intent	Require clarification or attach no-execution constraints.
Computational overload	Delegate, decompose, clarify, or deny before inference.
Non-replayable admission	Persist deterministic evidence and reason codes.

11. Implementation Mapping

AIKernel.NET can map this model to the following interface contracts.

```
namespace AIKernel.Abstractions.Governance.PreInference;

public interface IPreInferenceGovernancePipeline
{
    ValueTask<PreInferenceResult> ProcessAsync(
        IContextSnapshot context,
        CancellationToken cancellationToken = default);
}

public interface ICriticalOperationGate
{
    ValueTask<CriticalOperationGateResult> EvaluateAsync(
        CriticalOperationProfile profile,
        CancellationToken cancellationToken = default);
}

public interface IComputationalComplexityGate
{
    ValueTask<ComplexityGateResult> EvaluateAsync(
        TaskComplexityProfile taskProfile,
        ModelExecutionBudget budget,
        CancellationToken cancellationToken = default);
}
```

The detailed DTOs are implementation artifacts. The architectural requirement is that they remain immutable, versioned, serializable, and replayable.

12. Limitations

1. **Requires structured context.** This model assumes that context fragments have passed quarantine and structural validation.
2. **Requires deterministic validators.** Admissibility checks must not depend on probabilistic LLM calls. They must be implemented as deterministic code, policy rules, schemas, or versioned heuristics.
3. **Task extraction is not omniscient.** Task profiles are governance evidence, not ground truth. If a profile cannot be derived deterministically, the extractor must return `Unknown` and the pipeline must fail closed.
4. **Does not govern trajectories.** This paper determines whether inference may begin. Runtime convergence, semantic drift, tool-candidate risk, and post-generation control belong to Trajectory Governance.
5. **Does not implement full policy algebra.** ABAC or XACML-style policy evaluation may inform the PDP layer, but this paper defines the admission boundary and evidence model, not a complete enterprise authorization language.

13. Relationship to Other Phase-1 Papers

- **Paper 01** provides governed ROM knowledge and snapshot identity used as trusted context input.
- **Paper 02** provides storage capability facts and semantic storage boundaries used by capability admission.
- **Paper 04** monitors semantic trajectories after a request has been admitted.
- **Paper 05 and later papers** consume admission decisions as part of the AIKernel execution pipeline.

14. Conclusion

Pre-Inference Admissibility Governance does not make stochastic inference safe after the fact. It prevents inadmissible inference from starting.

By combining prompt-injection defense, capability admission, critical-operation control, and computational-complexity gating, AIKernel establishes a deterministic entry boundary for governed AI execution. The result is a fail-closed, replayable, and auditable admission-control model that transforms the initiation of stochastic inference into a verifiable system transition.

References

1. Hartmanis, Juris, and Richard E. Stearns. “On the Computational Complexity of Algorithms.” Transactions of the American Mathematical Society, vol. 117, 1965, pp. 285-306. DOI: 10.1090/S0002-9947-1965-0170805-7.
2. Sikka, Varin, and Vishal Sikka. “Hallucination Stations: On Some Basic Limitations of Transformer-Based Language Models.” arXiv:2507.07505, 2025. DOI: 10.48550/arXiv.2507.07505.
3. National Institute of Standards and Technology. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST Special Publication 800-162, 2014. DOI: 10.6028/NIST.SP.800-162.
4. OASIS. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard, 22 January 2013.
5. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
6. Sogawa, Takuya. “AIKernel Phase-1 Paper 01: ROM Format and Knowledge Snapshot Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20306539.
7. Sogawa, Takuya. “AIKernel Phase-1 Paper 02: VFS Architecture and Semantic Storage Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20307891.